Chapitre 1

Rappels

Structure machine

1- Définitions

On rappelle quelques définitions utiles :

- – un bit est une valeur binaire qui, par convention, peut prendre la valeur 0 ou 1 ;
- – un octet est une donnée codée sur 8 bits ;
- – un mot est une donnée codée sur 16 bits ;
- – un Koctet est un ensemble de 1024 octets.

2- Les constituants de base de la machine

Dans ce paragraphe, nous rappelons les principaux composants d'une machine.

a- Le processeur

Le processeur est le cœur de tout ordinateur : il exécute les instructions qui composent les programmes. Les instructions sont stockées en mémoire (en dehors du processeur). Ces instructions (dont l'ensemble compose le langage d'assemblage, ou assembleur) sont très simples mais n'en permettent pas moins, en les combinant, de réaliser n'importe quelle opération programmable.

Pour exécuter un programme, le processeur lit les instructions en mémoire, **une à une**. Il connait à tout instant l'adresse (l'endroit dans la mémoire) à laquelle se trouve la prochaine instruction à exécuter car il mémorise cette adresse dans son **compteur ordinal.**

Les instructions agissent sur des données qui sont situées soit en mémoire, soit dans des registres du processeur.

Un registre est un élément de mémorisation **interne au processeur** et contenant une valeur. Les registres sont en nombre très limité (14 pour le 8086). Pour accéder à une donnée en mémoire, il faut spécifier son adresse.

Pour accéder à une donnée dans un registre, il faut spécifier son nom (chaque registre possède un nom qui est une chaine de caractères).

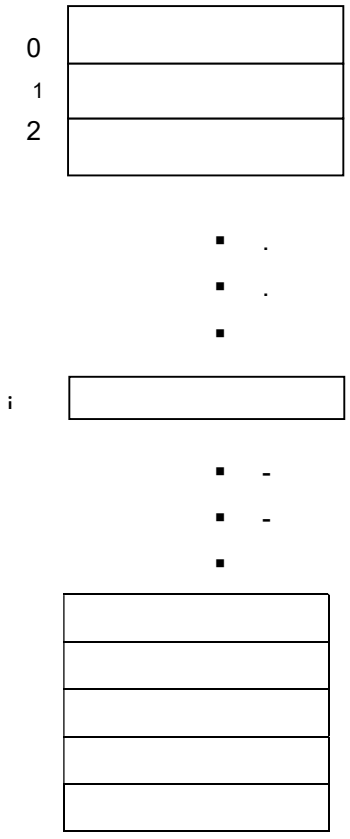
Le 8086 est un processeur 16 bits, c'est-à-dire qu'il traite des données codées sur 16 bits.

b- La mémoire

La mémoire est une séquence d'octets, tous numérotés de manière unique par un entier compris entre 0 et N −1. On dit alors que la capacité de la mémoire est de N octets. On a toujours N qui est une puissance de 2 $(N=2^m)$.

On appelle **adresse effective** (AE) d'un octet en mémoire le numéro qui lui est associé (voir .fig suivante).

Adresses effectives



Une mémoire de N octets.

Pour le 8086, un octet est désigné par un couple (numéro de segment, déplacement dans le segment) avec :

- Un segment est un ensemble de 64 Koctets consécutifs.
- ***Un déplacement (ou offset***) spécifie un octet particulier dans un segment .

Segment et déplacement sont codés sur 16 bits et peuvent donc prendre une valeur comprise entre 0 et 65535.

Remarque : On a une relation entre adresses effectives et couple (segment, déplacement) sous la forme de l'équation :

- **adresse effective = 16 × segment + déplacement**

Exemple:

Soit l'adresse physique **12345h** (hexadecimal), le registre **DS = 1230h** et le registre **SI = 0045h**. CPU calcule l'adresse physique en ,multipliant le contenu du registre segment par 10h et l'ajoute au contenu du registre SI (1230h * 10h + 45h = 12345h) car 16=10h.

Une conséquence de la relation énoncée plus haut entre adresse effective et adresse segmentée est qu'un octet d'adresse effective donnée peut être accédée de plusieurs manières. Plus précisément, à chaque AE correspond $2^{12} = 4096$ différentes.

Remarque :

En général, les registres **BX, SI** et **DI** sont combinés avec le registre **DS** ; **BP** et **SP** sont combinés avec le registre **SS** segment.

Lors de son exécution, un programme utilise plusieurs segments mémoire
– un segment contient les instructions du programme (le segment de code) ;
– un segment contient les données du programme (le segment de données) ;
– un segment contient la pile du programme (le segment de pile). Ce segment très important sera décrit ultérieurement

Ces trois segments peuvent se situer n'importe où en mémoire et même se recouvrir partiellement ou totalement. Leur attribution en mémoire est généralement réalisée

automatiquement par le système d'exploitation sans que le programmeur ait à s'en soucier. En respectant un certain protocole, on est certain de référencer le bon segment sans se poser de questions trop compliquées.

c- Les registres

Le 8086 possède 14 registres de 16 bits. Vu de l'utilisateur, le 8086 comprend 3 groupes de 4 registres de 16 bits, un registre d'état de 9 bits et un compteur programme de 16 bits non accessible par l'utilisateur.

a- Les registres généraux

Les registres généraux participent aux opérations arithmétiques et logiques ainsi qu'à l'adressage.
Chaque demi-registre est accessible comme registre de 8 bits, le 8086 peut donc effectuer des opérations 8 bits d'une façon compatible avec le 8080.

AX

AH	AL

BX

BH	BL

CX

CH	CL

DX

DH	DL

b- Registres d'adressage

SP

BP

SI

DI

c- Registres de segments

CS

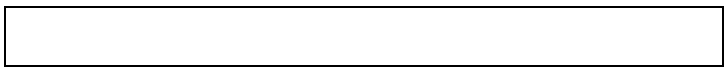
DS

SS

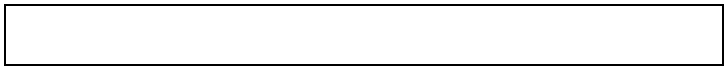
ES

d- Registres de commande

IP



FLAGS



Dans ce qui suit, nous allons détailler le rôle de chacun.

Registres généraux

AX : Accumulateur

- Usage général,
- Obligatoire pour la multiplication et la division,
- Ne peut pas servir pour l'adressage

BX : Base

- Usage général,
- Adressage, (Par défaut, son offset est relatif au segment DS)

CX : Comptage et calcul

- Usage général,
- Utilisé par certaines instructions comme compteur de répétition.
- Ne peut pas servir pour l'adressage

DX : Data

- Usage général,
- Dans la multiplication et la division 16 bits, il sert comme extension au registre AX

pour contenir un nombre 32 bits,

- Ne peut pas servir pour l'adressage

Registres d'adressage (offset)

Ces registres de 16 bits permettent l'adressage d'un opérande à l'intérieur d'un segment de 64 ko (2^{16} positions mémoires)

SP : Pointeur de Pile

- Utilisé pour l'accès à la pile. Pointe sur la tête de la pile.
- Par défaut, son offset est relatif à SS

BP : Pointeur de Base

- Adressage comme registre de base, (Par défaut, son offset est relatif à SS)
- Usage général

SI : Registre d'index (source)

- Adressage comme registre d'index, (Par défaut, son offset est relatif à DS)
- Certaines instructions de déplacement de données l'utilise comme index de l'opérande source. L'opérande destination étant indexé par DI
- Usage général

DI : Registre d'index (destination)

- Adressage comme registre d'index, (par défaut, son offset est relatif à DS)
- Certaines instruction de déplacement de donnés l'utilise comme index de l'opérande destination, l'opérande destination étant indexé par SI

Les registres de segment

Ces registrent sont combiné avec les registres d'offset pour former les adresses. Une case mémoire est repérée par une adresse de la forme RS:RO. On place le registre segment au début d'une zone mémoire de de 64Ko, ensuite on fait varier le registre d'offset qui précise l'adresse relative par rapport à cette position.

CS : **C**ode **S**egment

Définit le début de la mémoire programme. Les adresses des différentes instructions du programme sont relatives à CS

DS : Data Segment

Début de la mémoire de données dans laquelle sont stockées toutes les données traitées par le programme

SS : Stack Segment

Début de la pile.

La pile est une zone mémoire gérée d'une façon particulière. Elle est organisée comme une pile d'assiettes. On pose et on retire les assiettes toujours sur le haut de la pile. Un seul registre d'adresse suffit donc pour la gérer, c'est le stack pointer SP. On dit que c'est une pile LIFO (Last IN, First Out).

Empiler une donnée : sauvegarder une donnée sur (le sommet) de la pile

Dépiler une donnée : retirer une donnée (du sommet) de la pile

ES : Extra Segment

Début d'un segment auxiliaire pour données

Format d'une adresse

Une adresse doit avoir la forme [Rs : Ro] avec les possibilités

Rien	Valeur
ES	DI
CS	SI
ES	BP
DS	BX

Remarque 1

Si le registre segment n'est pas spécifié (cas rien), alors le processeur l'ajoute par défaut selon l'offset choisit :

Offset utilisé	valeur	DI	SI	BX	BP
Registre Segment par défaut qui sera utilisé par le CPU	DS				SS

Remarque 2

Dans ce cours, On accèdera souvent à la mémoire en précisant seulement la partie offset de l'adresse. Par défaut on considère qu'on est dans le segment DS

Remarque 3

Les valeurs des registres cs, ds et ss sont automatiquement initialisées par le système d'exploitation au lancement du programme. Ces segments sont alors implicites, c'est-à-dire que si l'on désire accéder à une donnée en mémoire, il suffit de spécifier son offset.

Le registre d'état (flags)

				O	D	I	T	S	Z		A		P		C

Six bits reflètent les résultats d'une **opération arithmétique ou logique** et 3 participent au control du processeur.

- **C** : (Carry) indique le dépassement de capacité de 1 sur une opération 8 bits ou 16 bits. Ce flag peut être utilisé par des instructions de saut conditionnel, des calculs arithmétique en chaîne ou dans des opération de rotation.

- **P** : (Parité) indique que le nombre de 1 est un nombre pair. Ce flag est utilisé avec certains sauts conditionnels.

- **A** : (retenue **A**rithmétique) indique une retenue sur les 4 bits (digit) de poids faible. Par exemple quand la somme des 2 digits de poids faible dépasse F (15)

- **Z** : (Zéro) Indique que le résultat d'une opération arithmétique ou logique est nul. Il est utilisé dans plusieurs instructions de sauts conditionnels.

- **S** : (Signe) reproduit le bit de poids fort d'une quantité signée sur 8 bits ou sur 16 bits. L'arithmétique signée fonctionne en complément à 2. S=0 : positif, S=1 : négatif. Ce flag sert lors de sauts conditionnels.

- **T** : (Trap) met le CPU en mode pas à pas pour faciliter la recherche des défauts d'exécution.

- **I** : (Interruption) autorise ou non la reconnaissance des interruptions

  - *I = 0* →*Interruptions autorisées*
  - *I = 1* →Interruptions non autorisées

- **D** : (Direction) fixe la direction de l'auto-inc/décrémentation de SI et DI lors des instruction de manipulation de chaînes.
  - *D = 0* → *Incrémentation des index*
  - *D = 1* → *décrémentation des index*

- **O** : (Overflow) indique un dépassement de capacité quand on travaille avec des nombres signés. Comme par exemple si la somme de 2 nombres positifs donne un nombre négatif ou inversement. (40h + 40h = 80h et O=1)

Chapitre 2

Codage des nombres

Les instructions du 8086 manipulent des valeurs numériques entières codées sur 1 ou 2 octets.
Nous décrivons ici le codage des nombres entiers.

Nous distinguons quatre types de codage :

- non signé pour des nombres entiers forcément positifs ;
- signé pour des nombres entiers positifs ou négatifs.
- décimal compacté, ou décimal codé binaire
- décimal non compacté

C'est au programmeur de décider du codage qu'il utilise et à écrire son programme en conséquence. L'ordinateur ne sait pas quel est le codage utilisé. Il exécute simplement des instructions.

Nous décrivons ces codages et quelques notions qui leur sont liées.
Nous supposerons dans ce qui suit que les données sont codées sur un octet (8 bits). Tout ce qui est dit peut-être adapté immédiatement pour des données qui seraient codées sur un mot, un double mot ou un nombre quelconque de bits.

1- Représentation non signée des entiers

Avec l bits, il est possible de coder 2^l valeurs différentes. Si $l = 8$, on peut donc coder 256 valeurs différentes. On peut établir le codage suivant, sur 8 bits :

Valeur	codage
0	00000000
1	00000001
2	00000010
3	00000011
…	…
255	11111111

Ce codage permet de représenter tout nombre entier entre 0 et 255 (0 et $2^l - 1$) avec 8 bits (l bits). Du fait que l'on ne représente que des nombres positifs, **ce codage est qualifié de non signé**.

2- Représentation signée des nombres entiers

Sur 8 bits, au lieu de représenter les nombres de 0 à 255, on représente les nombres de -128 à + 128 pour traiter les nombres signés.

Valeur	Codage
-128	10000000
-127	10000001
▪ ▪ ▪	▪ ▪ ▪ ▪ ▪ ▪ ▪
-45	11010011
▪ ▪ ▪	▪ ▪ ▪
-1	11111111
0	00000000
1	00000001
2	00000010
▪ ▪ ▪	▪ ▪ ▪
84	01010100
…	…
126	01111110
127	01111111

Ce codage se nomme codage par complément à deux. L'obtention du codage par complément à deux d'un nombre est indiqué plus bas.

On constate, ceci est très important pour que le processeur traite indifféremment des nombres en codage signé ou non, que le fait de faire une opération (par exemple ajouter 1 à une valeur) fonctionne quel que soit le codage utilisé. Faire la somme au niveau du codage binaire ou au niveau des nombres représentés donne le même résultat.

Dans le codage signé, on constate que le bit de poids fort vaut 1 pour tous les nombres négatifs, 0 pour tous les nombres positifs. Ce bit indique donc le signe du nombre codé. Aussi, ce bit se nomme le bit de signe. Il faut bien prendre garde que si le bit de signe vaut 0, la valeur représentée est la même en codage signé ou non signé. Par contre, si le bit de signe vaut 1, la valeur codée est supérieure à 127 en codage non signé, inférieure à 0 en codage signé.

3- Codage décimal

En codage décimal 1, un chiffre décimal (compris donc entre 0 et 9) est codé sur 4 bits. 4 bits codant en principe 16 valeurs différentes, seules 10 de ces 16 combinaisons sont effectivement utilisées en codage décimal, les autres n'ayant alors pas de sens. Le codage est le suivant :

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

On distingue deux types de représentation, **compactée ou** non. Un chiffre nécessitant 4 bits pour son codage.

Codage décimal non compacté

On peut représenter deux chiffres par octet. Ce codage porte le nom de codage décimal compacté.

Codage décimal non compacté

On également ne mettre qu'un seul chiffre par octet, les 4 bits de poids fort de l'octet étant inutilisés. On qualifie ce codage décimal non compacté.

Exemple

18 se code en décimal compacté 00011000

En décimal non compacté 00000001

00001000

NB : Deux octets sont nécessaires dans la représentation non compactée.

4- Complément à deux d'un nombre

Nous indiquons ici comment obtenir le complément à deux d'un nombre. Cette méthode donne la représentation binaire signée d'un nombre négatif.

L'algorithme est comme suit

.Soit à calculer la représentation du nombre $-n$ (où n est un nombre positif). On effectue les opérations suivantes :

- ✓ écrire n sous forme binaire
- ✓ complémenter tous les bits ($0 \rightarrow 1$, $1 \rightarrow 0$)
- ✓ ajouter la valeur 1 à ce nombre binaire.

Exemple. Soit à calculer le complément à deux de -23 :

- ✓ la représentation binaire sur 8 bits de 23 est 00010111
- ✓ par complément des bits, on obtient : 11101000
- ✓ en ajoutant 1, on obtient : 11101001

Le complément à deux de 00010111 est donc 11101001. La représentation binaire signée de -23 est donc **11101001.**

– Les segments :

L'octet d'adresse effective α peut être adressée de multiples façons, selon que l'on considère qu'il se trouve dans le segment s, s + 1, s + 2, ...

Dans chacun de ces cas, l'adresse segmentée de l'octet α est :

- (s, d),
- (s + 1, d − 16),
- (s + 2, d − 32),

...

Notons que chaque octet-mémoire appartient à $2^{12} = 4096$ segments différents.

Chapitre 3

Programmation assembleur

Un programme en assembleur a une structure bien particulière. Chaque ligne du programme assembleur comporte une instruction.

Chaque ligne est composée des champs suivants :

- le champ étiquette (peut être vide)
- le champ mnémonique (le nom de l'instruction)
- le champ opérande (les arguments de l'instruction), (peut être vide)
- le champ commentaire (peut être vide)

a- Une **étiquette** est un identificateur composé de lettres, chiffres et de caractères $, %, _ et ?

Quelques exemples d'étiquettes valides :

- *boucle*,
- *fin_de_tant_que,*
- ...

b- Un **mnémonique** est, généralement, composé uniquement de lettres. Voici quelques mnémoniques que nous retrouverons souvent dans les programmes assembleur:

- MOV
- CMP
- ADD
- ...

On pourra indifféremment écrire les mnémoniques avec des lettres minuscules ou majuscules.

c- Les **opérandes** d'une instruction sont les données à traiter. Elles sont soit sous la forme de valeurs constantes, soit l'adresse mémoire (l'emplacement en mémoire) où se trouve la donnée, soit le nom d'un registre contenant la donnée ou contenant l'adresse mémoire de la donnée.

d- Un **commentaire** commence par **;** (point virgule) et se termine en fin de ligne.

1- **Structure d'un programme en assembleur**

Un programme en assembleur a la forme suivante :

```
.MODEL small        ;   définit le modèle mémoire
.STACK 200h         ;  taille de la pile
.DATA               ; définition des données du programme
      donnee1     DB      ....
      donnee2     DW      ....
          …
.CODE               ; les instructions composant le programme
      ....
      ....
      ....
     ....
    mov ah,4ch ;
    int 21h
    ....
....
END                 ;  fin du programme source
```

Un programme en assembleur commencera donc, par convention, par un ensemble de **commentaires** indiquant l'auteur du programme, la date de réalisation du programme ainsi que son objectif.

. MODEL

Permet de choisir l'un des modèles mémoires suivants :
- *tyni* : CS et DS utilisent un même segment de 64ko.

- *small* : CS et DS utilisent un segment de 64 ko chacun.

- *medium* : CS utilise plusieurs segments de 64 Ko et DS utilise un seul segment de 64ko.

- *compact* : est l'inverse du modèle medium.

- *large* : CS et DS utilisent plusieurs segments de 64ko chacun.

- *hogue* : c'est le modèle large avec des structures de données qui dépassent les limites d'un seul segment.

. STACK

Permet de déterminer la taille de la pile . Elle est utilisée lors de l'usage d'une pile.

.DATA

Marque le début du segment de données. Cette section est utilisée uniquement pour déclarer des données s'il y en a.

.CODE

Marque le segment code (d'instructions)..

Les données

Dans cette section, nous présentons la manière de déclarer les données d'un programme assembleur.

1.1 Spécifier une donnée en mémoire (variable)

Il y a ***16 manières*** de déclarer une données en mémoire

- Etiquette
- [bx]
- [si]
- [di]
- Bx + étiquette
- bx + si + étiquette
- bx + di+ étiquette

- [bx + si]
- [bx + di]
- [bp + si]
- [bp + di]
- Bp + étiquette
- bp + si + étiquette
- bp + di+ étiquette
- si + étiquette
- di+ étiquette

Etiquette : indique l'étiquette d'une donnée définie dans la section .data du programme ou un déplacement est une constante codée sur 16 bits.

Notons au passage que l'on peut récupérer la valeur des adresses de début de segment de données et de code en utilisant les symboles

@data

@code

respectivement.

Remarque : Toutes les références à des opérandes en mémoire sont implicitement situées dans le segment dont le numéro est contenu dans le registre DS. Lorsque cela n'est pas le cas, cela sera mentionné explicitement.

1.2 Les constantes

a- Les constantes numériques

On peut écrire les constantes numériques dans les bases 2, 8, 10 et 16. Une constante numérique commence toujours par un chiffre.

Par défaut, une constante est notée en base 10.

En base 2, une constante ne doit comporter que des chiffres 0 et 1 et être terminée par la lettre b.

En base 8, une constante ne doit comporter que des chiffres 0, 1, 2, 3, 4, 5, 6 ou 7 et se terminer par la lettre o.

En base 16, une constante ne doit comporter que des chiffres de 0 à 9 ou des lettres de A à F pour les valeurs hexadécimales correspondant aux nombres décimaux 10, 11, 12, 13, 14 et 15 et se terminer par la lettre h.

Exemple :

- ✓ 100110b est une constante binaire dont la valeur décimale est 38
- ✓ 37o est une constante octale dont la valeur décimale est 31
- ✓ 4Ch est une constante hexadécimale dont la valeur décimale est 76

Remarque : Puisqu'une constante numérique doit toujours commencer par un chiffre, une constante hexadécimale dont le premier caractère est une lettre sera précédée d'un 0. Ainsi, la valeur C3 sera notée 0C3h en assembleur.

b- Les constantes chaines de caractères

Une chaine de caractères est une suite de caractères entre deux '. Si on veut mettre un caractère ' dans une constante, il suffit de la doubler.

Exemples :

– 'hello world'

– 'l''arbre'

2- Programmation en assembleur

Déclaration d'une constante

La directive **EQU** associe une valeur à un symbole qui pourra être ensuite utilisé dans le programme à la place de la constante qu'elle définit. Sa syntaxe est :

nom EQU constante

Exemple

La ligne : Quatre EQU 4

définit une constante qui s'appelle Quatre dont la valeur est 4. Une fois cette déclaration effectuée, toute occurrence de l'identificateur Quatre sera remplacée par la valeur 4.

déclaration de variables

La directive DB

[nom] DB constante [, constante]

Réserve et initialise un octet. nom est un symbole permettant d'accéder à cet octet.

Exemple

les lignes suivantes :

```
UN_OCTET DB 36
DEUX_OCTETS DB 43, 26
LETTRE DB 'e'
CHAINE DB 'hello world !', 13, 10, '$'
TABLEAU DB 10 dup (0)
```

définissent 5 symboles comme suit :

– NB_OCTET référence une donnée codée sur un octet dont la valeur sera initialisée à 36.

– DEUX_OCTETS référence une donnée codée sur un octet dont la valeur sera initialisée à 43, l'octet suivant étant initialisé à 26. On peut considérer que DEUX_OCTETS référence un tableau de 2 octets ;

– LETTRE référence une donnée dont la valeur sera initialisée avec le code ASCII du caractère ’e’.
– CHAINE référence une chaine de caractères composée des caractères ’h’, ’e’, ’l’, ’l’, ’o’, ’ ’, ’w’, ’o’, ’r’, ’l’, ’d’, ’ ’, ’!’, des caractères de code 13 (retour en début de ligne) et 10 (aller à la ligne suivante) et enfin du caractère ’$’ (les caractères de code 13 et 10 placés en fin de chaine entrainent le passage à la ligne après l’affichage du message ; le caractère ’$’ indique la fin de la chaine de caractères. En général, toute chaine de caractères devant être affichée à l’écran se terminera par un caractère ’$’.).

– TABLEAU référence un tableau de 10 octets dont la valeur initiale des éléments est 0.

Remarque : On constate donc que la valeur initiale d’un symbole peut être exprimée de différentes manières :
– une valeur
– plusieurs valeurs séparées par des virgules
- la notation [xxx] indique que xxx est facultatif

La directive DW

[nom] DW constante [, constante]
Réserve et initialise un mot (16 bits).
nom est un symbole permettant d’accéder à ce mot.

Exemples :

```
MOT    DW    1559
MOT2  DW    ((45 * 23) / 9 + 23)
TMOT  DW  20  DUP (0)
```

– le symbole MOT référence une donnée codée sur un mot et qui sera initialisée avec la valeur 1559 ;
– le symbole MOT2 référence une donnée codée sur un mot et qui est initialisée avec la valeur 138 ;
– le symbole TMOT référence un tableau de 20 mots initialisés avec la valeur 0.

Remarque
Signalons la différence entre un symbole défini par une directive EQU et un symbole défini par une directive DB ou DW. Si nous considérons les exemples précédents :
– Quatre est une constante dont la valeur est 4;
– OCTET, MOT, DEUX_OCTETS, ... sont des constantes dont la valeur est l'adresse à laquelle seront stockées les valeurs 36, 1559, 43, 26, ...

Par analogie avec un programme Pascal, nous pouvons dire que Quatre est une constante, alors que les symboles (OCTET, MOT, ...) sont des variables qui seront initialisées avec les valeurs indiquées. Au cours de l'exécution du programme, la valeur se trouvant à l'adresse OCTET pourra être modifiée.
Notons que les octets composant un mot sont stockés de la manière suivante : le poids faible est mis dans le premier octet, le poids fort dans le deuxième octet.

Exemple :

```
M DW 1234h
```

La valeur 34h est mise à l'adresse M et la valeur 12h à l'adresse M+1.

Chapitre 4

Instructions du processeur Intel 9086

Dans ce chapitre, nous présentons les principales instructions du microprocesseur 8086.

1- Les instructions Arithmétiques

Le 8086 permet d'effectuer les quatre opérations arithmétiques de base, l'addition, la soustraction, la multiplication et la division. Les opérations peuvent s'effectuer sur des nombres de 8 bits ou de 16 bits signés ou non signés. Les nombres signés sont représentés en complément à 2. Des instructions d'ajustement décimal permettent de faire des calculs en décimal.

- L' **Addition :**

ADD Od , Os

- **Od : opérande destination**
- **Os : opérande source**

L' instruction ADD additionne l'opérande source et l'opérande destination et met le résultat dans l'opérande destination, Od + Os → Od

On distingue les cas possibles suivants :

- ADD register /variable, registre
- ADD registre, registre/variable
- ADD registre/variable, constante

Exemple

- ADD AX, 123
- ADD AX, BX
- ADD [123], AX

- ADD BX, [SI]

Une autre variante de ADD est ADC

ADC Od , Os

- **Od : opérande destination**
- **Os : opérande source**

ADC additionne l'opérande source, l'opérande destination et le curry et met le résultat dans l'opérande destination, Od + Os + C → Od

L'incrémentation : Cette variante de ADD ajoute 1 à l'opérande.

INC Op

- **Op : opérande**

INC incrémente l'opérande Op, c'est à dire Op + 1 → Op

- **Attention**, l'indicateur C n'est pas positionné quand il y a débordement, c'est l'indicateur Z qui permet de détecter le débordement
- Pour incrémenter une case mémoire, il faut préciser la taille :

  - INC byte [ ]
  - INC word [ ]

Exemples

a- Cet exemple permet d'ajouter la valeur 7 au registre AX :

```
MOV AX, 0008h
ADD AX, 0007h
```

Ce programme retournera donc la valeur 000Fh dans le registre AX.

b- L'instruction suivante ajoute la valeur du registre BL au registre AL (AL = AL + BL) :

```
ADD AL , BL
```

c- L'instruction suivante ajoute la valeur hexadécimal 74 au registre CL (CL = CL + 74) :

ADD CL,74h

d- L'instruction suivante ajoute le registre AL au contenu d'un octet de l'emplacement mémoire du segment de données adressé par BX avec la somme entreposée dans le même emplacement mémoire :

ADD [BX],AL

e- L'instruction suivante ajoute le contenu des mots de l'emplacement mémoire du segment de données adressé par *SI+2* au registre *BX* avec la somme entreposée dans le registre BX :

ADD BX,[SI+2]

f- L'instruction ADD suivante permet d'ajouter le contenu du mot de l'emplacement mémoire du segment de données adressé par TEMP + DI au registre *BX* avec la somme entreposée dans BX :

ADD BX,Temp[DI]

e- L'instruction ADD suivante permet d'ajouter la valeur 3 au contenu d'un octet de l'emplacement de mémoire du segment de données adressé par DI avec la somme entreposée au même emplacement :

ADD Byte Ptr[DI],3

f- L'instruction ADD suivante a pour effet d'ajouter la valeur 7 au registre *AL* :

ADD AL,7

g- L'instruction ADD suivante ajoute la valeur 74 au registre *AX* :

ADD AX,74

h- ADD suivante ajoute la valeur 7 au registre *EAX* :

ADD EAX,7

i- L'instruction ADD suivante a pour effet d'ajouter la valeur -2 au registre *RAX* :

ADD RAX,-2

j- L'instruction ADD suivante a pour effet d'ajouter la valeur hexadécimal 7A à l'emplacement mémoire *EAX* :

ADD [EAX],07Ah

k- L'instruction ADD suivante a pour effet d'ajouter la valeur du registre *RSI* à l'emplacement mémoire *RAX* :

ADD [RAX],RSI

l- L'instruction ADD suivante a pour effet d'ajouter la valeur de l'emplacement mémoire *RAX* au registre *RAX* :

ADD RAX,[RAX]

m- L'instruction ADD suivante a pour effet d'ajouter la valeur de l'emplacement mémoire *RAX* au registre *EAX* :

ADD EAX,[RAX]

n- L'instruction ADD suivante a pour effet d'ajouter la valeur 74 au symbole *UnSymbole* :

ADD [UnSymbole],74

o- L'instruction ADD suivante a pour effet d'ajouter la valeur du registre *EAX* au symbole *UnSymbole* :

ADD [UnSymbole],EAX

Remarque:

La retenue est positionnée en fonction du résultat de l'opération.

Par exemple, si les instructions suivantes sont exécutées ;

```
MOV ax, a9h
ADD ax, 72h
```

alors le registre AX contient la valeur 1bh, le bit C est positionné à la valeur 1, la retenue auxiliaire A est mise à 0.

Si nous considérons les deux instructions suivantes ;

```
MOV ax, 09h
ADD ax, 3ah
```

alors le registre AX contient la valeur 43h, le bit C est mis à 0, la retenue auxiliaire A est mise à 1.

La Soustraction

SUB **Od , Os**

Cette instruction Soustrait l'opérande source et l'opérande destination avec résultat dans l'opérande destination.

Od - Os → Od

Une variante de SUB est SBB. Elle Soustrait l'opérande source et le curry de l'opérande destination avec résultat dans l'opérande destination.

Od - Os - C → Od

SBB **Od , Os**

Remarque

La retenue est positionnée en fonction du résultat de l'opération.

Exemple

Soient les instructions suivantes :

mov ax, 39h
sub ax, 18h

le registre AX contient ensuite la valeur 21h. Les bits Z, S et C du registre FLAGS sont mis à 0 car le résultat n'est pas nul, son signe est positif et aucune retenue n'est générée.

Considérons maintenant les deux instructions suivantes ;

mov ax, 26h
sub ax, 59h

le registre AX contient la valeur cdh. Le bit Z est mis à zéro. Les bits C, A et S sont mis à 1.

- **Multiplication**

MUL **Op**

MUL est une instruction à un seul opérande. Elle effectue une multiplication non signée entre **l'accumulateur** (AL ou AX) et l'opérande Op. Le résultat de taille double est stocké dans l'accumulateur et son extension (AH:AL ou DX:AX).

MUL Op_8 → AL x Op_8 → AX
MUL Op_{16} → AX x Op_{16} → DX:AX

Remarques

- L'opérande Op ne peut pas être une donnée, c'est soit un registre soit une position mémoire, dans ce dernier cas, il faut préciser la taille (byte ou word)

MUL BL ; AL x BL → AX

MUL CX ; AX x CX → DX:AX

MUL byte [BX] ; AL x (octet pointé par BX) → AX

MUL word [BX] ; AX x (word pointé par BX) → DX :AX

- Les deux instructions IMUL et MUL effectuent des multiplications.
- L'instruction IMUL effectue la multiplication d'opérandes signés. L'instruction MUL effectue la multiplication d'opérandes non signés.

Les indicateurs de retenue (C) et de débordement (O) sont mis à 1 si le résultat ne peut pas être stocké dans l'opérande destination.

Une addition ou une soustraction de données codées sur n bits donne un résultat sur au plus n + 1 bits. Le bit supplémentaire est la retenue et est stocké dans le bit C de flags.

Par contre, une multiplication de deux données de n bits donne un résultat sur 2n bits. Dans leur première forme qui prend une donnée 8 bits en opérande (donc le résultat est sur 16 bits),

les instructions MUL et IMUL effectuent le produit de la valeur contenue dans le registre al avec la valeur de l'opérande fourni. Le résultat est placé dans le registre ax.

Dans leur deuxième forme qui prend une donnée 16 bits en opérande (donc le résultat est sur 32 bits),

les instructions MUL et IMUL effectuent le produit de la valeur contenue dans le registre ax avec la valeur de l'opérande fourni. Le résultat est placé dans la paire de registres dx et ax. dx contient le poids fort du résultat, ax le poids faible.

Exemple de multiplication sur 8 bits

Soient les instructions suivantes :

```
mov al, 4
mov ah, 25
imul ah
```

A l'issue de l'exécution de ces 3 instructions, le registre AH contient la valeur 100, produit de 4 par 25.

Pour bien comprendre la différence entre les instructions imul et mul, regardons les exemples suivants :

```
mov bx, 435
mov ax, 2372
imul bx
```

A l'issue de l'exécution de ces 3 instructions, ax contient la valeur be8c et dx la valeur f, soit la valeur hexadécimale fbe8c, c'est-à-dire 1031820, le produit de 435 par 2372. Les deux données étant positives, le résultat est le même que l'on utilise l'instruction imul ou l'instruction mul.

Considérons maintenant la séquence d'instructions :

```
mov bx, -435
mov ax, 2372
imul bx
```

A l'issue de leur exécution , ax contient la valeur 4174 et dx contient la valeur fff0, soit la valeur -1031820. Si l'on remplace imul par mul, le résultat n'a pas de sens

- **Division**

  DIV **Op**

DIV Effectue la division AX/Op8 ou (DX|AX)/Op16 selon la taille de Op qui doit être soit un registre soit une mémoire.

Dans le dernier cas, il faut préciser la taille de l'opérande,

Exemple :

DIV byte [adresse] ou DIV word [adresse].

DIV Op8 ; AX / Op8 , Quotient → AL , Reste → AH

DIV S16 ; DX:AX / S16 , Quotient → AX , Reste → DX

Exemple

```
mov ax, 37
mov dx, 5
div dl
```

Après exécution, le quotient égal à 7 est dans AL et AH contient le reste de la division 2.

Exemple

```
mov bx, 435
mov ax, 2372
div bx
```

A l'issue de l'exécution de cette séquence d'instructions, le registre ax contiendra la valeur 5 qui est le quotient de 2372 par 435, et le registre dx vaudra 00C5h qui est le reste de la division.

Remarques

- S ne peut pas être une donnée (immédiat)
- Après la division l'état des indicateurs est indéfini
- La division par 0 déclenche une erreur

IDIV Op Identique à DIV mais effectue une division signée

CBW (Convert Byte to Word)

CBW Effectue une extension de AL dans AH. On écrit le contenu de AL dans AX en respectant le signe.

Si AL contient un nombre positif, On complète par des 0 pour obtenir la représentation sur 16 bits.
Si AL contient un nombre négatif, On complète par des 1 pour obtenir la représentation sur 16 bits.

Exemple :

+5 = 0000 0101 → 0000 0000 0000 0101

-5 = 1111 1011 → 1111 1111 1111 1011

CWD (Convert Word to Double Word) effectue une extension de AX dans DX en respectant le signe. On écrit AX dans le registre 32 bits obtenu en collant DX et AX DX | AX

2- Les instructions booléennes et logiques

Ces instructions disponibles sur tous les processeurs travaillent sur les données au niveau des bits (et non sur des valeurs numériques comme les instructions vues jusqu'à présent).

a- AND (et logique)

```
AND registre / variable, registre
AND registre , registre/variable
AND registre / variable, constant
```

L'opérateur AND réalise un et-logique bit à bit entre l'opérande source et l'opérande destination. Le résultat est rangé dans l'opérande destination.

Exemple

Considérons les deux lignes suivantes :

```
mov al, 36h
and al, 5ch
```

Le registre AL contient ensuite la valeur 14h obtenue de la manière suivante ;

36h= 0011 0110

5ch= 0101 1100

Le résultat du AND est 00010100

Les bits S et Z sont mis à zéro et le bit P à 1.

b- OR logique

OR registre / variable, registre
OR registre , registre/variable
OR registre / variable, constant

L'opérateur OR réalise un OR -logique bit à bit entre l'opérande source et l'opérande destination. Le résultat est rangé dans l'opérande destination.

Exemple

Considérons les deux lignes suivantes :

```
mov al, 36h
and al, 5ch
```

Le registre AL contient ensuite la valeur 14h obtenue de la manière suivante ;

36h= 0011 0110

5ch= 0101 1100

Le résultat du OR est 01111110

Les bits S et Z sont mis à zéro et le bit P à 1

c- NOT Négation logique

NOT registre/variable

NOT transforme la valeur d'un registre ou d'un opérande mémoire en son complément logique bit à bit.

Exemple

Considérons les deux lignes suivantes :

```
mov al, 36h
not al
```

Le registre AL contient ensuite la valeur c9h obtenue de la manière suivante :

¬ 36h= ¬ 0011 0110
c9h= 1100 1001

Les bits S et P sont mis à 1, le bit Z à 0.

d- XOR ou-exclusif

XOR registre/variable, registre
XOR registre, registre/variable
XOR registre/variable, constante

L'opérateur XOR réalise un ou-exclusif bit à bit entre l'opérande source et l'opérande destination. Le résultat est rangé dans l'opérande destination.

Exemple

Considérons les deux lignes suivantes :

```
mov al, 36h
xor  al, 5ch
```

Le registre AL contient ensuite la valeur 6ah obtenue de la manière suivante :

```
36h    =  0011 0110
⊕ 5ch  =  0101 1100
6ah    =  0110 1010
```

Les bits S et Z sont mis à zéro et le bit P à 1

3- Les instructions de transfert

a- MOV transfert d'une valeur

Ces instructions réalisent des transferts de données entre 2 adresses mémoire, 2 registres, ou entre un registre et la mémoire. Ceci correspond donc à l'affectation des langages de haut niveau A:= B.

```
MOV registre/variable, registre
MOV registre, registre/variable
MOV registre, registre de segment
MOV registre de segment, registre
MOV registre/variable,  constant
```

- registre de segment indique l'un des registres cs, ds, es ou ss.
- l'instruction MOV effectue le transfert d'une donnée vers un registre ou une adresse mémoire.

Remarque ;Pour le transfert de la valeur d'un registre de segment vers un autre registre de segment : aucun mode d'adressage de la commande MOV ne permet de transférer la valeur d'un registre de segment dans un autre registre de segment. Nous disposons uniquement de la possibilité de transférer la valeur d'un registre de segment vers un registre de données et de la possibilité de transférer la valeur d'un registre de données vers un registre de segment. Aussi, pour atteindre notre but, nous devrons effectuer le transfert en deux é tapes, en utilisant un registre de données intermédiaires :

- transférer la valeur du registre de segment source dans un registre de données sur 16 bits

- transférer la valeur de ce registre de données dans le registre de segment destination

Exemple

Supposons que nous voulions transférer le contenu du registre DS dans le registre ES. Nous pouvons l'écrire sous la forme suivante :

```
mov ax, ds
mov es, ax
```

b- Chargement de l'offset d'une donnée

```
LEA registre/variable, étiquette
```

L'instruction LEA charge l'offset de la donnée référencée dans le deuxième opérande qui est, en général , une donnée déclarée dans le segment de données.

c- XCHG Echange de valeurs

```
XCHG registre/variable, registre
XCHG registre, registre/variable
```

L' instruction XCHG échange le contenu de deux emplacements mémoire ou registres.

Exemple

Supposons que le registre AX contienne la valeur 20 et le registre DX la valeur 10, l'exécution de l'instruction

```
xchg ax, dx
```

entraînera : AX contient la valeur 10 et DX la valeur 20.

3. Les tests en assembleur

En assembleur, il n'existe pas de tests comme dans les langages de haut niveau tel Pascal ou C. Cependant, il est possible de réaliser des tests. On utilise pour cela les bits du registre FLAGS comme condition de test et une instruction de branchement conditionnelle (saut si certains bits du registre FLAGS valent 0 ou 1) pour déclencher la partie ***alors*** ou la partie **sinon** du test.

a- Principe

Les bits du registre FLAGS sont positionnés par les instructions que nous avons déjà vues (instructions arithmétiques, logiques, ...). Ils sont également positionnés par des instructions spécialement conçues pour réaliser des tests et qui n'ont d'autres effets que de positionner les bits de FLAGS en fonction de certaines conditions sur leurs opérandes. Ces instructions sont ***CMP et TEST.***

Une fois les indicateurs positionnés, une instruction dite de saut conditionnel teste un bit ou une combinaison de bits de FLAGS et, en fonction du résultat :

- effectue une rupture de séquence (un saut) vers un endroit précis dans le code où l'exécution se poursuit normalement.
- continue en séquence si le test ne donne pas un résultat positif.

Nous présentons l'instruction CMP, les instructions de sauts conditionnels et inconditionnels puis présentons sur des exemples le codage des tests.

b- Les instructions de comparaison

b-1 L'instruction CMP pour les comparaisons

C'est l'instruction la plus utilisée pour positionner les indicateurs avant d'effectuer une instruction de saut conditionnel.

CMP registre/variable, registre
CMP registre, registre/variable
CMP registre/variable, constante

CMP permet de comparer deux valeurs. Pour cela CMP soustrait le second opérande du premier, sans cependant modifier l'opérande destination, mais en positionnant les indicateurs en fonction du résultat.

.
Ainsi, si le résultat de la soustraction est nul, donc l'indicateur Z a été positionné à 1, cela signifie que les deux valeurs comparées sont égales.

En utilisant des raisonnements du même genre, on peut savoir si les deux valeurs sont différentes, ordonnées strictement ou non.

A l'issue de l'exécution des deux instructions suivantes :

```
mov al, 23
cmp al, 34
```

le registre AL n'est pas modifié par l'exécution de l'instruction CMP et contient toujours la valeur 23 affectée auparavant . L'indicateur de retenue C est positionné à 1, ce qui indique que le deuxième opérande de l'instruction CMP est supérieur à la valeur du premier opérande. L'indicateur de zéro Z valant 0, cela indique que les deux données sont différentes (sinon, la soustraction d'un nombre à lui-même donne 0, donc le bit Z est positionné à 1). Le bit de signe S est également mis à 1 car 29 − 34 est un nombre négatif.

b-2 Jxx les instructions de saut conditionnels

LES TESTS EN ASSEMBLEUR

Toutes les instructions de saut conditionnel prennent le même type d'opérande (attention, bien distinguer la valeur zéro 0 du bit O du registre d'état) :

JA	étiquette	saut si supérieur (C =0 et Z =0)
JAE	étiquette	saut si supérieur ou égal (C =0)
JB	étiquette	saut si inférieur (C =1)
JBE	étiquette	saut si inférieur ou égal (C =1 ou Z =1)
JC	étiquette	saut si retenue (C =1)
JCXZ	étiquette	saut si CX vaut 0
JE	étiquette	saut si égal (Z =1)
JG	étiquette	saut si supérieur (Z =0 ou S =O)
JGE	étiquette	saut si supérieur ou égal (S =O)
JL	étiquette	saut si inferieur (S 6=O)
JLE	étiquette	saut si inférieur ou égal (Z =1 ou S 6=O)
JNC	étiquette	saut si pas de retenue (C =0)
JNE	étiquette	saut si non égal (Z =0)
JNO	étiquette	saut si pas de débordement (O =0)
JNP	étiquette	saut si pas de parité (P =0)
JNS	étiquette	saut si pas de signe (S =0)
JO	étiquette	saut si d´débordement (O =1)
JP	étiquette	saut si parité (P =1)
JS	étiquette	saut si signe (S =1)

Toutes ces instructions fonctionnent selon le schéma suivant. Quand la condition est vraie, un saut est effectué à l'instruction située à l'étiquette spécifiée en opérande.

Notons que les tests d'ordre (inférieur, supérieur, ...) se comprennent de la manière suivante. Supposons que nous comparions deux données par une instruction CMP et que nous exécutions ensuite une instruction

JG, c'est-à -dire '' saut si plus grand''. Il y aura alors saut si la valeur du deuxième opérande de l'instruction CMP est supérieur à la valeur du premier opérande.

L´étiquette référencée dans l'instruction de saut conditionnel ne doit pas se trouver trop loin de l'instruction de saut. Sinon, une erreur d'assemblage est déclenchée et le message :

Relative jump out of range by xxx bytes

est affiché. Pour remédier à cette erreur, il faut procéder de la manière suivante. Supposons que l'instruction:

```
je étiquette
```

provoque l'erreur en question. Il faut transformer cette instruction dans la séquence d'instructions :

```
    jne nouvelle étiquette
    jmp étiquette
nouvelle-étiquette:
```

C'est-à -dire :

1. remplacer l'instruction de saut conditionnel par l'instruction de saut correspondant à la condition opposée (ici, saut si le bit Z vaut 1 est remplacé par saut si le bit Z vaut 0). L'opérande de cette nouvelle instruction est une nouvelle étiquette.

2. faire suivre l'instruction de saut conditionnel par une instruction de saut inconditionnel jmp dont l'opérande est l'étiquette de l'instruction de saut conditionnel originale

3. déclarer ensuite la nouvelle étiquette après l'instruction jmp

b-3 JMP : saut inconditionnel

JMP étiquette

L'instruction JMP effectue un saut inconditionnel à l'étiquette spécifiée. Contrairement à un saut conditionnel, le saut est toujours effectué, le registre FLAGS n'intervenant en rien dans cette opération

.

Exemples de codage de tests en assembleur

Généralement parlant, un test dans un langage évolué a la forme suivante :

SI (condition vraie) ALORS
action-alors
SINON
action-sinon
FIN_SI

Si la valeur de l'expression condition est vraie, les instructions composant la partie action-alors sont exécutées. Sinon, les instructions de la partie action-sinon sont exécutées.

En assembleur, ce type de construction est réalisé à l'aide du squelette de programme suivant :

calcul de la condition
Jcc SINONn
action-alors
...
JMP FSIn
SINONn:

```
        action-sinon
        ...
FSIn:
        ...
```

Où Jcc dénote l'une des instructions de saut conditionnel vues plus haut.

Nous notons les points suivants :
– le calcul de la condition positionne les indicateurs du registre FLAGS
– en fonction de ce positionnement, un saut conditionnel est réalisé par l'instruction Jcc (où cc représente la condition à tester – voir les différentes possibilités dans la section traitant des instructions de saut conditionnel) vers la partie sinon du test. Si la condition est vérifiée, aucun saut n'est réalisé et le programme poursuit son exécution en séquence avec les instructions de la partie action-alors

– une fois les instructions de la partie alors exécutées, un saut doit être réalisé (instruction JMP FSIn) pour atteindre la fin de la structure de test afin de ne pas également exécuter la partie sinon

– une étiquette (SINONn) est déclarée indiquant le début de la partie action-sinon où le branchement conditionnel doit être exécuté pour déclencher l'exécution des instructions de la partie sinon du test

– une fois la partie sinon exécutée, on continue en séquence avec les instructions situées après le test (c'est-à -dire, après l'étiquette FSIn)

Pour des raisons de clarté, nous utiliserons un format spécial pour les étiquettes codant un test. Une étiquette SINON indique le début de la séquence action-sinon et une étiquette FSI indique la fin de la structure de test. Ces étiquettes seront numérotées, les tests étant numérotés dans le programme de 1 à n, par convention.

Afin d'illustrer le propos, nous donnons un exemple de codage d'un test. Nous voulons effectuer la division d'un nombre par un autre. La division n'est possible que si le diviseur est non nul. Aussi, nous devons tester si l'opérande diviseur est nul avant d'exécuter la division. Si le diviseur est nul, nous désirons que le résultat du traitement soit -1. Sinon, le résultat est le quotient des deux nombres.

```
.data
        val db .... ; mettre ici la valeur du diviseur


.code
            ...
        mov ax, ... ; valeur du dividende
```

```
        cmp val, 0
        je SINON1 ; diviseur nul => saut
         mov bl, [val]
         div bl
         jmp  FSI1
SINON :
        mov ax, -1 ; diviseur nul => résultat vaut -1
FSI1: ...
         ...
```

Notons que ce programme n'est pas complet. Il vise simplement à montrer le codage d'un test.

c- Les boucles en assembleur

Comme pour les tests, il n'existe pas de structure générale en assembleur pour coder une boucle. Cependant, à l'aide des instructions vues précédemment pour réaliser des tests, on peut programmer n'importe quel type de boucle. On verra également que des instructions simplifient grandement la programmation de la boucle pour.

c.1 Principe

Le principe de la réalisation des boucles tant-que ou jusqu'a est assez particulier. Une boucle est essentiellement composée de deux éléments :

– une **condition** de sortie qui indique quand la boucle doit être interrompue, qu'il faut sortir de la boucle et continuer l'exécution des instructions en séquence

– **un corps de boucle** qui spécifie l'action à réaliser pendant que la condition de sortie n'est pas vérifiée à chaque itération de la boucle

La condition de sortie va donc être codée d'une manière ressemblant aux tests. Un saut conditionnel testera cette condition et entrainera, quand la condition est vérifiée, la sortie de la boucle.

Le corps de la boucle devra pour sa part ✯✯ boucler ✯✯, c'est-à-dire, une fois exécuté, entrainera le re-calcul de la condition de sortie et la prise de décision quant à la poursuite ou non des itérations
.
Nous indiquons maintenant comment est réalisé le codage d'une boucle **tant-que** et d'une boucle **répéter**.

c.1.1 **Boucle tant-que**

Le squelette d'une boucle tant-que, dans un langage de haut niveau, est le suivant :

```
TANT-QUE (condition) FAIRE
    action
FIN_TQ
```

Cela va se traduire en assembleur sous la forme suivante :

```
TQn: calcul de la condition
     Jcc FTQn
     action
     ...
    JMP TQn
    FTQn:
...
```

où Jcc dénote l'une des instructions de saut conditionnel vues plus haut.

Nous notons les points suivants :

– une étiquette TQ indique le début de la boucle

– la boucle commence par une évaluation de la condition de sortie qui positionne les indicateurs du registre FLAGS

– en fonction de la valeur des indicateurs (donc du résultat du test de sortie), un saut conditionnel est effectué en fin de boucle (Jcc FTQn), pour quitter la boucle le moment venu

– on trouve ensuite le corps de la boucle, terminé par une instruction de saut inconditionnel vers le début de la boucle (JMP TQn) qui permettra de ré-évaluer la condition d'arrêt après chaque itération
– une étiquette FTQ indiquant la fin de la boucle

Pour des raisons de clarté, nous utiliserons un format spécial pour les étiquettes codant une boucle

TANT-QUE. Une étiquette TQ indique le début de la structure de boucle et une étiquette FTQ indique la fin de la structure de boucle. Ces étiquettes seront numérotées, les boucles étant numérotées dans le programme de 1 à n, par convention.

c.1.2 Boucle répéter

Le squelette d'une boucle REPETER dans un langage de haut niveau est le suivant :

```
REPETER
        action
JUSQUA (condition vraie)
```

Cela se traduit de la manière suivante en assembleur :

```
REPETERn:
        action
        ...
      calcul de la condition
     Jcc REPETERn
```

Notons les points suivants :

– une étiquette REPETER repère le début de la boucle

– on trouve ensuite le corps de la boucle

– à l'issue de l'exécution du corps, on trouve l'évaluation du test d'arrêt qui positionne les indicateurs du registre FLAGS

– une instruction de saut conditionnel effectue un branchement pour continuer les itérations si la condition d'arrêt n'est pas vérifiée

Pour des raisons de clarté, nous utiliserons un format spécial pour les étiquettes codant une boucle REPETER. Une étiquette REPETER indique le début de la boucle. Ces étiquettes seront numérotées,

les boucles REPETER étant numérotées dans le programme de 1 à n, par convention.

c.1.3 Les boucles pour

Une boucle pour est généralement codée à l'aide d'une instruction de la famille LOOP.

- LOOP boucles contrôlées par un compteur
- LOOP étiquette saut court si le compteur est différent de 0
- LOOPE étiquette saut court si compteur différent de 0 et Z =1
- LOOPNE étiquette saut court si compteur différent de 0 et Z =0

LOOP fonctionne avec le registre CX qui joue le rôle de compteur de boucles. LOOP décrémente le compteur sans modifier aucun des indicateurs. Si le compteur est différent de 0, un saut à l'étiquette opérande de l'instruction LOOP est réalisé. Le squelette d'une boucle pour est comme suit :

```
POUR indice := 1 A bs FAIRE
        action
  FAIT
```

Cependant, en assembleur, seules existent des boucles ayant un indice variant d'une certaine valeur bs à 1, en décrémentant sa valeur à chaque itération. Aussi, la boucle précédente doit initialement être transformée dans une boucle du genre :

```
POUR indice := bs A 1, pas := -1 FAIRE
    action
FAIT
```

Cette boucle se traduit en assembleur de la manière suivante :

```
    mov cx, bs
POURn: ...
    action
    loop POURn
```

En exemple, nous écrivons un programme qui affiche vingt fois la chaine de caractères 'hello world'.

```
        .data
        msg: db 'hello world', 13, 10, '$'
        .code
        mov ax, @data ; initialisation de la valeur
```

```
mov ds, ax ; de ds
mov cx, 20 ; initialisation du compteur de boucles
;
; corps de boucle
;
boucle: mov ah, 9 ; affichage du message
lea dx, msg
int 21h
loop boucle ; contrôle de la boucle
```

Le début initialise le registre DS pour qu'il contienne le numéro du segment de données et le compteur de boucles. Le corps de la boucle contient alors un appel à la routine affichant un message à l'écran. Il est très courant de parcourir un tableau dans une boucle tout en voulant sortir de la boucle lorsqu'une condition est vérifiée (parce que l'on cherche une valeur dans le tableau possédant une propriété particulière par exemple).

Notons que ce type de construction n'existe pas en Pascal. Des instructions sont disponibles en assembleur pour réaliser ce genre de boucle, les instructions LOOPE et LOOPNE (les instructions LOOPZ et LOOPNZ étant des synonymes). Comme pour les boucles LOOP, le registre CX joue le rôle de compteur de boucle. Par ailleurs, à chaque itération, une condition (la valeur du bit Z du registre FLAGS) est testée. Pour l'instruction LOOPE, il y a saut à l'étiquette spécifiée en opérande tant que le compteur est non nul et que l'indicateur Z vaut 1. Pour l'instruction LOOPNE, il y a saut à l'étiquette spécifiée en opérande tant que le compteur est non nul et que l'indicateur Z vaut 0.

Exemple

Nous présentons un exemple de boucle ainsi que les conventions que nous adoptons pour le codage d'une boucle pour. Nous recherchons dans un tableau d'entiers l'élément d'indice le plus faible qui soit multiple de 7.

```
.data
      tableau db 34, 72, 48, 32, 56, 12, 8, 9, 45, 63, 80
.code
      mov ax, @data
      mov ds, ax
      mov cx, 11 ; nombre d'éléments dans la table
      mov dl, 7 ; valeur du diviseur
      mov bx, 0 ; indice de l'élément considéré
  BOUCLE1:
```

```
mov ah, 0
mov al, BYTE PTR table+bx
div dl ; division de l'élément courant par 7
inc bx ; mise à jour de l'indice
cmp ah, 0 ; ah contient le reste de la division
loopne BOUCLE1 ; saut si non multiple et non terminé
```

A l'issue de l'exécution de ces instructions, le registre BX contient l'indice du premier élément du tableau table ayant une valeur multiple de 7.

On notera les points suivants :

– une étiquette BOUCLE1 indique le début de la boucle ;

– on trouve ensuite le corps de la boucle ;

– une instruction (ici CMP) positionne les indicateurs du registre FLAGS pour éventuellement sortir de la boucle avant d'avoir effectué toutes les itérations (quand une valeur multiple de 7 a été trouvée, on sort immédiatement) ;

– l'instruction LOOPNE décide s'il faut boucler en fonction de la position des indicateurs du registre FLAGS et de la valeur du compteur de boucle (le registre CX) qui est automatiquement décrémenté.
Comme pour les autres structures, on notera que les étiquettes BOUCLE sont numérotées de 1 à n.

3.3.5 Instructions diverses

Nous décrivons dans cette section quelques instructions qui se rév`elent souvent utiles pour le codage des tests ou des boucles.

Instructions diverses

Nous décrivons ici quelques instructions ne prenant aucun opérande et agissant chacune sur un bit particulier du registre d'indicateurs FLAGS.

CLC mise à zéro de l'indicateur de retenue C

CLD mise à zéro de l'indicateur de direction D

CMC complémente l'indicateur de retenue C

STC mise à 1 de l'indicateur de retenue C

STD mise à 1 de l'indicateur de direction D

Le traitement des chaînes de données

Nous présentons dans cette section des instructions spécialement conçues pour le traitement de séquences de données, que ces données soient des caractères (des octets) ou des mots. Chacune effectue le traitement d'un élément de chaîne. Il y a cinq types d'instruction :

– LODS : chargement d'un élément de chaîne depuis la mémoire dans un registre

– STOS : écriture d'un élément de chaîne en mémoire

– MOVS : transfert d'un élément entre deux chaînes

– CMPS : comparaison entre deux éléments de chaîne

– SCAS : comparaison entre une valeur et un élément de chaine

Toutes ces instructions sont conçues pour être aisément insérées dans une boucle qui traitera l'ensemble des éléments de la chaine. Classiquement, lors d'une itération, un élément d'une chaine est référencé par son index dans la chaine. Pour pouvoir itérer l'instruction, il faut donc incrémenté cet index après chaque traitement. Pour nous aider, les instructions de traitement de chaînes incrémentent automatiquement l'index.

Par ailleurs, ces instructions traitent des chaînes d'octets, ou de mots. Aussi, selon le type de données traitées, 1 ou 2 doit être ajouté à la valeur de l'index. C'est effectivement de cette manière qu'agissent ces instructions.

Notons également que pour traiter les chaînes de leur fin vers leur début plutôt que dans le sens contraire, il suffit de soustraire 1 ou 2 à la valeur de l'index. Pour cela, l'indicateur D du registre FLAGS indique la direction du traitement. Si D vaut 0, le traitement va du début vers la fin de la chaine et la valeur 1 ou 2 est ajoutée à l'index. Si D vaut 1, le traitement va dans le sens contraire et la valeur 1 ou 2 est soustraite à l'index.

Notons que le bit D est positionné par les instructions STD et CLD. Remarque importante : les indicateurs de FLAGS ne sont pas modifiés par les opérations modifiant la valeur des index.
Dans les instructions de traitement de chaînes de caractères, le registre SI pointe implicitement vers la chaine source, le registre DI vers la chaine destination.

LODS chargement d'un élément de chaine

LODSB

LODSW

LODSB charge le registre AL (respectivement, AX pour l'instruction LODSW) avec l'octet (respectivement le mot) pointé par le registre SI. Une fois le chargement effectué, il est automatiquement ajouté 1 (respectivement 2) à SI si l'indicateur D vaut 0, ou retiré 1 (respectivement 2) si l'indicateur D vaut 1.

MOVS transfert d'un élément entre deux chaînes

MOVSB
MOVSW

MOVSB copie l'octet (respectivement le mot pour l'instruction MOVSW) pointé par SI vers l'octet (respectivement le mot) pointé par ES:DI. Le segment destination est forcément ES. Par contre, le segment de la source qui est DS par défaut peut être redéfini.

Une fois la copie réalisée, SI et DI sont automatiquement modifiés. Si l'indicateur de direction vaut 0, 1 (respectivement 2) est ajouté à SI et DI. Si l'indicateur de direction vaut 1, 1 (respectivement 2) est retiré à SI et DI.

STOS écriture en mémoire d'un élément d'une chaine

STOSB
STOSW

STOSB écrit en mémoire l'octet (respectivement le mot pour l'instruction STOSW) se trouvant dans le registre AL (respectivement dans le registre AX) en mémoire à l'adresse pointée par le registre DI. Le segment destination est forcément ES.
Une fois la copie réalisée, DI est automatiquement modifié. Si l'indicateur de direction vaut 0, 1 (respectivement 2) est ajouté à DI. Si l'indicateur de direction vaut 1, 1 (respectivement 2) est retiré à DI.

Exemple : multiplier par 2 tous les éléments d'un tableau d'entiers

```
.data
    TABLEAU DW 10, 20, 30, 40, 50, 60, 70, 80, 90, 100
    RES DW 10 DUP (0)
.code
    mov ax, @data
    mov ds, ax
    mov es, ax
```

```
    lea si, TABLEAU
    lea di, RES
    mov cx, 10
BOUCLE1:
     lodsw
     mov bl, 2
    mul bl
    stosw
loop BOUCLE1
```

Exemple

Un deuxième exemple illustre la transformation d'une table de données codées en décimal non compacté en ASCII. Cette opération est simplement obtenue par ou-logique avec la valeur 30h (le code ASCII des chiffres 0, 1, ... 9 est 30h, 31h, ... 39h).

```
.data
    TABLEAU DW 1, 2, 3, 4, 5, 6, 7, 8, 9
     RES DW 9 DUP (0)
.code
    mov ax, @data
    mov ds, ax
    mov es, ax
    lea si, TABLEAU
    lea di, RES
    mov cx, 9
BOUCLE1:
    lodsw
    or al, 30h
    stosw
   loop BOUCLE1
   …
```

CMPS comparaison de chaînes

CMPSB
CMPSW

CMPSB compare l'octet (respectivement le mot pour l'instruction CMPSW) pointé par SI à l'octet (respectivement le mot) pointé par ES:DI. Le segment destination est forcément ES. Par contre, le segment de la source qui est DS par défaut peut être redéfini.
Une fois la copie réalisée, SI et DI sont automatiquement modifiés. Si l'indicateur de direction vaut 0, 1 (respectivement 2) est ajouté à SI et DI. Si l'indicateur de direction vaut 1, 1 (respectivement 2) est retiré à SI et DI.

SCAS comparaison d'un élément de chaine avec une valeur

SCASB
SCASW

SCASB compare la valeur de l'octet (respectivement le mot pour l'instruction SCASW) contenu dans le registre AL (respectivement AX) à l'octet (respectivement le mot) pointé par ES:DI. La donnée en mémoire est toujours référencée via le registre de segment ES.

Une fois la comparaison réalisée, DI est automatiquement modifié. Si l'indicateur de direction vaut 0, 1 (respectivement 2) est ajouté à DI. Si l'indicateur de direction vaut 1, 1 (respectivement 2) est retiré à DI.

REP
REPE
REPZ
REPNE
REPNZ répéter une instruction de traitement de chaînes de données

Nous décrivons ici une instruction qui permet d'itérer sur toute une chaine les opérations élémentaires vues précédemment. Cette instruction préfixe en fait l'opération à itérer. Elle donne tout son intérêt aux instructions LODS, STOS, MOVS, CMPS, SCAS.

REP ne peut préfixer que les instructions LODS, STOS, MOVS. Un compteur (registre CX) indique le nombre d'itérations à effectuer.

Les préfixes REPE et REPNE ne peuvent préfixer que les instructions CMPS et SCAS. Pour les préfixes

REPE et REPNE, un compteur (registre CX) indique le nombre maximal d'itérations à effectuer. Par ailleurs, l'instruction élémentaire est répétée tant qu'une condition (égalité ou non-égalité entre les opérandes de CMPS ou SCAS) n'est pas remplie.

REP MOVSB
REP MOVSW
REP LODSB
REP LODSW
REP STOSB
REP STOSW
REPE CMPSB
REPE CMPSW
REPE SCASB
REPE SCASW
REPNE CMPSB
REPNE CMPSW
REPNE SCASB
REPNE SCASW

– REP MOVS copie CX éléments de la chaine pointée par SI vers la chaine pointée par ES:DI
– REP LODS charge CX éléments de la chaine pointée par SI dans AL (ou AX)
– REP STOS écrit CX éléments de la chaine pointée par ES:DI avec la valeur contenue dans AL (ou AX)
– REPE CMPS compare au plus CX éléments de la chaine pointée par ES:DI avec ceux de la chaine pointée par SI. Les itérations sont poursuivies tant que les éléments des deux chaînes sont égaux et tant que le compteur n'est pas nul. D`es que l'une de ces conditions n'est plus vérifiée, l'instruction REPE CMPS est terminée
– REPE SCAS compare au plus CX éléments de la chaine pointée par ES:DI avec la valeur du registre AL, ou AX selon le cas. Les itérations sont poursuivies tant que les éléments de la chaine sont égaux à la valeur du registre et tant que le compteur n'est pas nul. D`es que l'une de ces conditions n'est plus vérifiée, l'instruction REPE SCAS est terminée
– REPNE CMPS compare au plus CX éléments de la chaine pointée par ES:DI avec ceux de la chaine pointée par SI. Les itérations sont poursuivies tant que les éléments des deux chaînes sont différents et tant que le compteur n'est pas nul. D`es que l'une de ces conditions n'est plus vérifiée, l'instruction REPNE CMPS est terminée
– REPNE SCAS compare au plus CX éléments de la chaîne pointée par ES:DI avec la valeur du registre AL, ou AX, selon le cas. Les itérations sont poursuivies tant que les éléments de la chaîne sont différents de la valeur du registre et tant que le compteur n'est pas nul. D`es que l'une de ces conditions n'est plus vérifiée, l'instruction REPNE SCAS est terminée

Notons que REPZ est un synonyme de REPE ; REPNZ est un synonyme de REPNE.

Pour résumer, à chaque itération, les actions suivantes sont exécutées :

1. tester le compteur (CX). S'il est à zéro, sortir de l'instruction de répétition et déclencher l'exécution de la suivante. Sinon :
2. exécuter l'opération sur la chaîne
3. décrémenter le compteur
4. si l'opération est SCAS ou CMPS, tester l'indicateur de zéro. Si la condition d'itération n'est pas remplie, poursuivre avec l'instruction qui suit le REP. Sinon :
5. recommencer

Exemple

Copie d'un tableau de caractères dans un autre tableau.

```
.data
    CHAINE db 'hello world' ; chaîne source
    RES db 11 dup (?) ; chaıne cible
.code
    mov ax, @data
    mov ds, ax
    mov es, ax
    lea di, RES ; offset chaıne cible
    lea si, TABLE ; offset chaıne source
    mov cx, 11 ; longueur de la chaıne
    rep movsb ; copie
    ...
```

Exemple

Recherche d'un caractère dans une chaîne de caractères en utilisant l'instruction SCAS

```
.data
    CHAINE db 'hello world'
.code
     mov ax, @data
     mov es, ax
     mov al, 'w'
     lea di, CHAINE
     mov cx, 11
     repne scasb
...
```

A l’issue de l’exécution de cette séquence d’instructions, le registre DI pointe sur le caractère qui se trouve après le ’w’ dans la chaîne de caractère, c’est-à-dire ’o’. Puisque l’offset de CHAINE vaut 0, di vaut 7 à l’issue de l’exécution de repne scasb.

Exemple

Recherche d’une chaîne de caractères dans une autre. à l’issue de cette recherche, le registre AX indiquera le résultat et vaudra 1 si la chaîne est trouvée, 0 sinon.

```
.data
        CH1 db ’hello world’
        CH2 db ’rl’
.code
        mov ax, @data
        mov ds, ax
        mov es, ax
        lea si, CH1
        mov ax, 1
        mov bx, 10
TQ1:    lea di, CH2
        mov cx, 2
        repe cmpsb
        je FTQ1 ; sous-cha^ıne trouvée
        dec bx
        jne TQ1
        mov ax, 0
        FTQ1: ...
```